

Code Audit Report

Module: Accounting & Owner Portal

Date: June 15, 2026

Target: /src/lib/accounting & /src/app/api/owner-portal

CRITICAL ISSUES

4

High risk of data mismatch or runtime failure

WARNINGS

3

Logical flaws and bypasses of type check

OBSOLETE FILES

1

Fully unused files recommended for removal

FILES AUDITED

2

Scope restricted to designated directories



Critical Findings Summary

The audit revealed severe flaws in both the inactive accounting utilities and the active owner-portal API route. Key problems include: hardcoded currency conversion rates which assume all non-USD currencies match AED 1:1, database ownership model mismatches bypassing fractional/contract-based property ownership table, in-memory aggregation of bookings without date boundaries presenting severe performance/scalability threat, and double-entry rounding math that can create unbalanced base-currency ledger transactions.

journal.ts

src/lib/accounting/journal.ts

DEAD / OBSOLETE FILE

FILE SIZE	LINES OF CODE	STATUS
3.66 KB (3,660 bytes)	131 lines	Dead (0 imports found)
RISK PROFILE		
High (if activated)		

This file was designed to handle the creation of double-entry journal entries and associated lines inside database transactions. However, it is currently ****completely unused**** throughout the codebase. If this module is to be activated in the future, it requires significant refactoring as it contains severe logical and syntax defects.

Findings Table

FINDING TYPE	DESCRIPTION	SEVERITY	LINES
Dead Code	The entire file, its interfaces (JournalLineInput, JournalEntryInput), and its primary function (createJournalEntry) are not imported or referenced anywhere in the active codebase.	• LOW	1 - 131
Date Deserialization Crash	input.entryDate.toISOString() expects a native JS Date object. When invoked from REST API payloads, dates are received as string values. The function does not	• HIGH	79

FINDING TYPE	DESCRIPTION	SEVERITY	LINES
	parse these inputs, causing a runtime crash (TypeError).		
Base Currency Mismatch	Base currency debits and credits are computed and rounded per-line via <code>toFixed(2)</code> . In multi-line foreign currency postings, individual rounding differences accumulate, resulting in unbalanced base debits and credits that violate double-entry integrity.	HIGH	109 - 110
Entry Number Race Condition	Generates sequential entry numbers (JE-YYYYMMDD-XXXX) using <code>tx.journalEntry.count()</code> . Simultaneous transactions will obtain matching counts, causing unique sequence collisions under concurrent write load.	MEDIUM	80 - 86
Type-Safety Bypass	Bypasses TypeScript type checks via the <code>// @ts-nocheck</code> directive, hiding type compiler warnings.	LOW	1

Code Highlights & Critical Locations

```

src/lib/accounting/journal.ts                                TypeScript

77: }
78: // Autogenerate an entry number
79: const todayStr = input.entryDate.toISOString().slice(0, 10).replace
80: const count = await tx.journalEntry.count({

81:   where: {
82:     organizationId: orgId,
83:     entryDate: input.entryDate,
84:   },

```

```
85: });
86: const entryNumber = `JE-${todayStr}-${String(count + 1).padStart(4,
...
107:   lines.map(async (line) => {
108:     const account = accountMap.get(line.accountCode!);
109:     const debitBase = Number((line.debitAmount * exchangeRate).toFi
110:     const creditBase = Number((line.creditAmount * exchangeRate).to
111:
112:     return tx.journalLine.create({
```

route.ts

src/app/api/owner-portal/revenue/route.ts

CRITICAL ACTION
REQUIRED

FILE SIZE	LINES OF CODE	STATUS
3.29 KB (3,370 bytes)	103 lines	Active (GET Endpoint)
RISK PROFILE		
Critical		

This endpoint aggregates financial bookings for property owners to display in their dashboards. It performs real-time financial math inside a tenant RLS transaction context. This file has multiple severe flaws, including hardcoded currency assumptions, a model lookup mismatch, and a performance scalability threat that will cause slow loading times as historical data accumulates.

Findings Table

FINDING TYPE	DESCRIPTION	SEVERITY	LINES
Broken Currency Math	Assumes any currency that is not "USD" is equivalent to "AED" (applying a 1.0 conversion factor). If the platform receives bookings in currencies like EUR, GBP, or OMR, their value is summed at 1:1, rendering calculations incorrect.	• HIGH	69 - 71
Ownership Model Mismatch	Filters properties using <code>propertiesFilter.ownerContactId = ownerContact.id</code> . However, the schema uses the <code>OwnerProperty</code> join table to handle property ownership. Querying the direct property field ignores property assignments mapped in the join table.	• HIGH	36 - 38
Performance Threat	Queries <i>all</i> bookings of statuses <code>confirmed</code> , <code>checked_in</code> , or <code>checked_out</code> for the owner's properties without date-range bounds. As booking counts grow, loading everything in memory will trigger severe database load and API timeouts.	• HIGH	44 - 58
Unused Import	The Prisma client instance (<code>prisma</code>) is imported from <code>@/lib/db/client</code> but is never referenced in the entire route code.	• LOW	3
Redundant Magic Number	The USD conversion rate (3.6725) is hardcoded twice across lines 70 and 89, rather than being encapsulated as a shared configuration constant.	• LOW	70, 89

FINDING TYPE	DESCRIPTION	SEVERITY	LINES
Type-Safety Bypass	Suppressing warnings via <code>// @ts-nocheck</code> on line 1 prevents typescript compilation diagnostics from catching type errors.	LOW	1

Code Highlights & Critical Locations

```

src/app/api/owner-portal/revenue/route.ts                                TypeScript

35:
36:   if (ownerContact) {
37:     propertiesFilter.ownerContactId = ownerContact.id; // Mismatch
38:   }
...
43:     include: {
44:       bookings: {
45:         where: {
46:           status: { in: ['confirmed', 'checked_in', 'checked_out']
47:         },
...
67:     for (const booking of prop.bookings) {
68:       const amount = Number(booking.grossAmount);
69:       // Standard conversion factor: USD to AED is 3.6725
70:       const factor = booking.currency.toUpperCase() === 'USD' ? 3
71:       grossRevenueAED += amount * factor;
72:     }

```

Recommendations & Action Plan

Suggested fixes to resolve findings



Implement Multi-Currency Support

Refactor `route.ts` to fetch real-time or daily currency exchange rates from a configuration table or external integration, instead of hardcoding USD to AED as `3.6725` and defaulting all other currencies to `1.0`.



Correct Property Ownership Lookup

Modify the property query filter in `route.ts` to search for matches in the `OwnerProperty` join table rather than only relying on the direct `Property.ownerContactId` relation, ensuring fractional owners are correctly mapped.



Add Date Boundaries & Pagination

Introduce `startDate` and `endDate` query parameters in the `GET /api/owner-portal/revenue` endpoint. Enforce database-level date range filters on bookings to restrict the queried volume, optimizing latency and avoiding timeouts.



Clean Up Dead Code & Obsolete Modules

Remove the unused `journal.ts` file if it is obsolete. If double-entry bookkeeping will be implemented in the future, rewrite the base-currency conversion code to balance debits and credits on a journal level instead of rounding each line separately. Also, replace `// @ts-nocheck` with strict TypeScript type signatures and remove the unused `prisma client` import from `route.ts`.